

Measuring Causal Effect with ARDL-BART

A non-parametric distributed-lag model with
Bayesian Additive Regression Trees

Theory, evidence, and a hands-on Python tutorial

Companion package: `pip install bartardl` ▪  github.com/merwanroudane/bartardl

After Mahdavi, Ehsani, Ahelegbey & Mohammadpour (2024), *Applied Mathematics* **15**, 292–312.

 Motivation & the ARDL set-up

 BART theory

sum-of-trees, priors, MCMC

 The competitors

LASSO, Elastic Net, Bayesian network

 Simulation evidence

 Macroeconomic application

 Applied lab with bartardl

 Guidance & take-aways

The one-sentence summary

Replace the *linear* conditional mean of an ARDL equation by a **sum of Bayesian regression trees**, so lag effects may be non-linear and interactive — then benchmark it honestly against penalised-regression selectors.

Part I

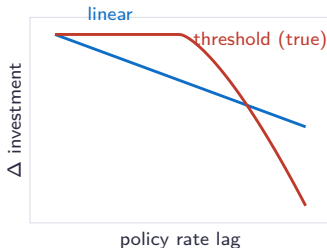
Motivation & the ARDL set-up

Why go non-linear?

Traditional macro models — VAR, dynamic factor, linear projection — share one limitation:

- they impose a **linear, additive** response surface;
- no threshold effects (e.g. policy rates), no interactions, no regime changes;
- misspecification $\Rightarrow$ biased forecasts and misleading policy read-offs.

Macro relationships are widely believed to be **non-linear**. We want a model that *learns* the shape of f from data.



A linear fit misses the kink where investment collapses once rates cross a threshold.

The ARDL equation — “a VAR, one equation at a time”

Single-equation ARDL

For a target y and a system of K variables $x_1, \dots, x_K$,

$$y_t = f(y_{t-1}, \dots, y_{t-p}, x_{1,t-1}, \dots, x_{K,t-p}) + e_t, \quad e_t \sim N(0, \sigma^2).$$

Every variable enters at lags $1, \dots, p$; equations are estimated **one at a time** (no cross-equation error covariance).

Classical ARDL: f is *linear* $\Rightarrow$ OLS.

This lecture: f is *unknown* and estimated **non-parametrically with BART**.

⚠ Naming, kept honest

Here “ARDL” means the distributed-lag *regression* (no bounds test / no error-correction term), and “causal effect” is operationalised as *forecasting* + *variable importance* — exactly what

Part II

BART theory

The sum-of-trees model

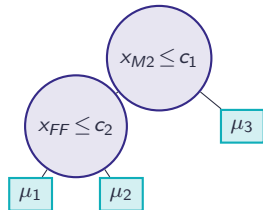
Bayesian Additive Regression Trees (Chipman, George & McCulloch, 2010)

$$y = f(\mathbf{x}) + \varepsilon, \quad \varepsilon \sim N(0, \sigma^2), \quad f(\mathbf{x}) \approx \sum_{l=1}^m g(\mathbf{x}; T_l, M_l).$$

Each T_l is a binary *decision tree* (split rules); $M_l = \{\mu_{l1}, \dots\}$ are its *leaf parameters*. The ensemble is a sum of m **weak learners**.

💡 Boosting, but Bayesian

Like boosting, BART adds many shallow trees; *unlike* boosting, a **prior** keeps each tree small and a **posterior** quantifies uncertainty in f and σ .



one tree $g(\mathbf{x}; T_l, M_l)$: follow the rules to a leaf value

Three priors that do the regularising

1. Tree structure

A node at depth d splits with probability

$$\alpha (1 + d)^{-\beta}.$$

Defaults $\alpha = 0.95$, $\beta = 2$ make deep trees very unlikely $\Rightarrow$ shallow, weak trees.

2. Leaf value

With y scaled to $[-\frac{1}{2}, \frac{1}{2}]$,

$$\mu \sim N(0, \tau), \quad \tau = \left(\frac{0.5}{k\sqrt{m}} \right)^2.$$

$k=2$: the m leaves sum to cover the data range in $\pm k$ SDs.

3. Error variance

$$\sigma^2 \sim \text{InvGamma}\left(\frac{\nu}{2}, \frac{\nu\lambda}{2}\right),$$

λ set so $\Pr(\sigma < \hat{\sigma}) = q$ from an OLS pilot. Defaults $\nu=3$, $q=0.9$.

💡 Regularisation prior

No single tree may explain much: the depth prior shrinks trees, the leaf prior shrinks μ toward 0. Signal is built up *additively* across the ensemble — this is why BART

Posterior sampling: Bayesian back-fitting MCMC

Gibbs sweep

For each tree $j = 1, \dots, m$ compute the *partial residual*

$$R_j = y - \sum_{h \neq j} g(\mathbf{x}; T_h, M_h),$$

update (T_j, M_j) given R_j , then draw σ^2 from its full conditional.

1. **Structure move** on T_j : a Metropolis–Hastings *grow* or *prune* proposal (accept/reject).
2. **Leaf draw**: each μ from its conjugate normal posterior $\mu \mid R_j \sim N(\text{mean}, 1/\text{prec})$,
 $\text{prec} = 1/\tau + n_{\text{leaf}}/\sigma^2$.
3. **Noise draw**: $\sigma^2 \sim \text{InvGamma}(\frac{\nu+n}{2}, \frac{\nu\lambda + \sum e^2}{2})$.

💡 Efficiency

Keep a running total fit and each tree's contribution: the partial residual is an $O(n)$ update, not a full re-evaluation of the ensemble.

`pip install bartardl`

The grow / prune acceptance ratio

Choosing split variable and value *uniformly* makes the rule probability cancel, leaving a compact log-acceptance for a **grow** at depth d :

Log acceptance (grow); b leaves before, w_2 nog-nodes after

$$\log A = \underbrace{\log p_d + 2 \log(1 - p_{d+1}) - \log(1 - p_d)}_{\text{tree prior}} + \underbrace{\log \frac{1 - p_g}{p_g} + \log \frac{b}{w_2}}_{\text{transition}} + \underbrace{\text{LL}_L + \text{LL}_R - \text{LL}_P}_{\text{marginal likelihood}}$$

with $p_d = \alpha(1 + d)^{-\beta}$ and, for a node of n residuals summing to s ,

$$\text{LL}(n, s) = \frac{1}{2} \log \frac{\sigma^2}{\sigma^2 + n\tau} + \frac{\tau s^2}{2\sigma^2(\sigma^2 + n\tau)}.$$

Prune is the exact reverse move. The leaf value μ is integrated out in LL, which is what makes the trees mix well.

Variable importance: inclusion proportions

Inclusion proportion

The share of *all* splitting rules, across post-burn-in draws and all m trees, that use predictor x_j :

$$\widehat{\text{IP}}_j = \frac{\#\{\text{splits on } x_j\}}{\#\{\text{splits}\}}, \quad \sum_j \widehat{\text{IP}}_j = 1.$$

A large $\widehat{\text{IP}}_j \Rightarrow x_j$ is repeatedly useful for partitioning the response $\Rightarrow$ an “important” driver.

In package terms:

```
model.inclusion_proportion_.
```



Reading it well

With *many* noise predictors, BART concentrates its splits on the few relevant ones — detecting low-dimensional structure in a high-dimensional lag matrix (here $8 \times 4 = 32$ regressors).

Part III

The competitors

Three “black-box” benchmarks

LASSO

$$\min_{\beta} \frac{1}{2} \|y - X\beta\|^2 + \lambda \|\beta\|_1$$

L1 penalty $\Rightarrow$ sparse selection. CV picks λ .

Elastic Net

$$+ \lambda \left(\alpha \|\beta\|_1 + \frac{1-\alpha}{2} \|\beta\|_2^2 \right)$$

Handles correlated groups; $\alpha=1$ is LASSO, $\alpha=0$ is ridge.

Bayesian network

Spike-and-slab / SSVS:
 $\gamma_j=1$ is a present edge
 $x_j \rightarrow y$. Posterior $\mathbb{E}[\gamma_j]$
 $=$ inclusion probability.



The SSVS sampler, in words

Draw each β_j from its normal full conditional; flip γ_j from a Bernoulli comparing slab vs. spike density at the current β_j ; draw σ^2 . Edges present most often are the “network” of drivers.

Part IV

Simulation evidence

Design: the Friedman (1991) data-generating processes

$\mathbf{x} \sim \text{Uniform}(0, 1)^p$; only $x_1, \dots, x_5$ matter, the rest are noise.

Non-linear DGP

$$y = 10 \sin(\pi x_1 x_2) + 20(x_3 - \frac{1}{2})^2 + 10x_4 + 5x_5 + \varepsilon$$

Linear DGP

$$y = 2 + 3(x_1 + x_2 + x_3 + x_4 + x_5) + \varepsilon$$

$p \in \{10, 100\}$; metric = *relative* RMSE
(RMSE / σ ; 1.0 is perfect).



The hypothesis

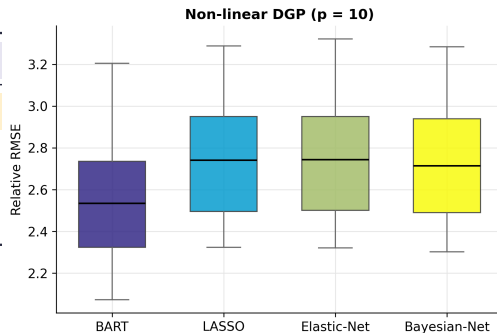
BART should **win** when f is **non-linear** and be merely *competitive* when f is truly linear (where a linear penalty is unbeatable).

Result 1 — non-linear DGP: BART wins

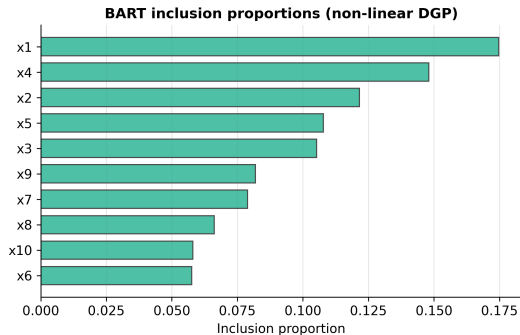
Relative RMSE, $p = 10$

method	mean	median	Q.50	Q.75
BART	2.554	2.534	2.534	2.735
LASSO	2.750	2.741	2.741	2.951
Elastic-Net	2.753	2.743	2.743	2.951
Bayesian-Net	2.727	2.714	2.714	2.939

BART attains the lowest error at every quantile.



Result 1 — and it finds the right variables



Low-dimensional structure

Inclusion proportions pile up on $x_1, \dots, x_5$; the noise predictors $x_6, \dots, x_{10}$ get almost no splits.



Reproduce it

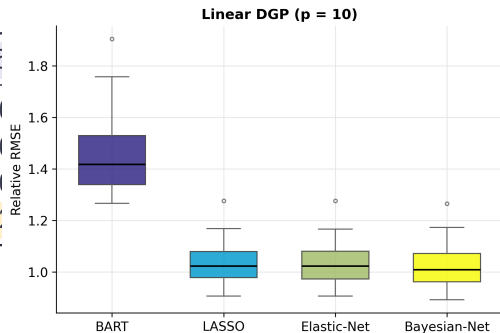
```
python  
examples/02_simulation_nonlinear.py
```

Result 2 — linear DGP: the linear selectors win

Relative RMSE, $p = 10$

method	mean	median	Q.50	Q.75
BART	1.474	1.418	1.418	1.530
LASSO	1.033	1.023	1.023	1.080
Elastic-Net	1.033	1.023	1.023	1.080
Bayesian-Net	1.023	1.009	1.009	1.074

BART is competitive but not best — exactly as expected.



💡 Honest science

A method that claimed to win *everywhere* would be suspicious. BART's edge is specifically **non-linearity**.

Part V

Macroeconomic application

Data: eight US series, quarterly 1959–2022

Paper Table 3 — variables and stationarity transforms

ID	FRED	code	description
GDP	GDPC1	5	real gross domestic product
INF	CPIAUCSL	5	consumer price index
FF	FEDFUNDS	2	federal funds rate
M2	M2SL	5	money stock M2
PC	PCECC96	5	real personal consumption
IP	INDPRO	5	industrial production
U	UNRATE	2	unemployment rate
INV	GPDIC1	5	real private investment

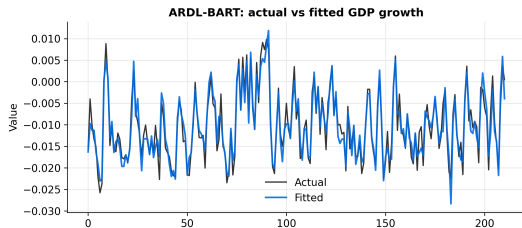
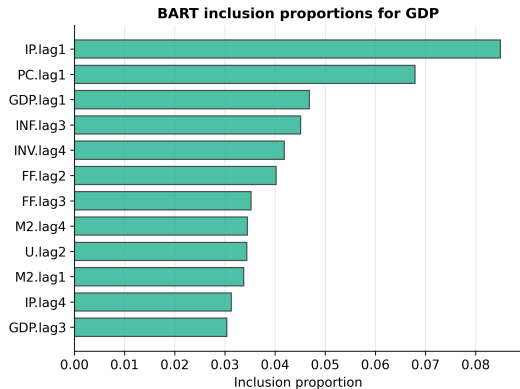
code 2 = first difference, 5 = first difference of log. $\text{ARDL}(4) \Rightarrow 8 \times 4 = 32$ regressors per equation.

Result 3 — forecast RMSE by target (Table 4)

target	BART	LASSO	Elastic-Net	Bayesian-Net
GDP	0.0032	0.0057	0.0057	0.0103
INF	0.0036	0.0037	0.0037	0.0075
FF	0.1216	0.0895	0.0905	0.0937
M2	0.0035	0.0031	0.0031	0.0076
PC	0.0039	0.0056	0.0056	0.0091
IP	0.0037	0.0065	0.0065	0.0099
U	0.1051	0.1251	0.1252	0.1367
INV	0.0118	0.0149	0.0149	0.0162

BART ranks #1 for 6 of 8 targets; the linear selectors take the two (near-)linear series (FF, M2).

Result 3 — what drives GDP?



💡 Interpretation

BART reads GDP growth off a handful of lags (consumption, industrial production, money, and own lags), with a close in-sample fit.

Part VI

Applied lab with bartard1

Install & the estimator protocol

```
pip install bartardl           # numpy, scipy, pandas, scikit-learn, matplotlib
```

One interface for every model

BART, Lasso, ElasticNet, BayesianNetwork all share:

```
est.fit(X, y)           # X: (n, p) array, y: (n,) array    -> self
est.predict(Xnew)       # -> (m,) array
est.importance_         # (p,) weights; BART: inclusion_proportion_
```

So any of them can be dropped into `ARDLBART(estimator=...)`.

Quick start — one ARDL-BART equation

```
import bartardl as ba

data = ba.load_macro()                # stationary US macro panel (8 vars)
model = ba.ARDLBART(
    n_lags=4,
    estimator=ba.BART,
    estimator_kwargs=dict(n_trees=100, n_burn=500, n_draws=500, random_state=0),
)
res = model.fit(data, target="GDP")

print("In-sample RMSE:", ba.rmse(res.y_train, res.fitted))
print(res.top_features(5))            # 5 most influential lagged regressors
```

```
In-sample RMSE: 0.0019
feature variable lag importance
PC.lag1      PC    1    0.0892
IP.lag1      IP    1    0.0760
GDP.lag1     GDP   1    0.0601
```

Simulation in five lines

```
draws = {"BART": [], "LASSO": []}
for rep in range(100):
    tr = ba.friedman(n=100, p=10, kind="nonlinear", random_state=rep)
    te = ba.friedman(n=100, p=10, kind="nonlinear", random_state=1000+rep)
    for name, est in [("BART", ba.BART(random_state=0)), ("LASSO", ba.Lasso())]:
        est.fit(tr.X, tr.y)
        draws[name].append(ba.relative_rmse(te.y, est.predict(te.X), 1.0))

print(ba.simulation_table(draws))      # -> Tables 1-2 layout
```

Match the paper exactly

```
ba.BART(n_trees=200, k=2, nu=3, q=0.9, alpha=0.95, beta=2,
n_draws=20000, n_burn=250)
```

The full horse-race + ranking table

```
data = ba.load_macro(); H = 40; train = data.iloc[:~H]
methods = {
    "BART": (ba.BART, dict(n_trees=200, n_draws=1000, random_state=0)),
    "LASSO": (ba.Lasso, {}),
    "Elastic-Net": (ba.ElasticNet, {}),
    "Bayesian-Net": (ba.BayesianNetwork, dict(n_iter=2000, burn=1000, random_state=0)),
}
rmse_by_target = {}
for tgt in data.columns:
    rmse_by_target[tgt] = {}
    for name, (est, kw) in methods.items():
        m = ba.ARDLBART(n_lags=4, estimator=est, estimator_kwargs=kw).fit(train, tgt)
        pred = m.predict(data, tgt).iloc[~H:]
        rmse_by_target[tgt][name] = ba.rmse(data[tgt].loc[pred.index], pred)

print(ba.ranking_table(rmse_by_target))           # -> Table 4
```

Publication figures & the Parula palette

```
from bartardl import viz
from bartardl.colors import parula_colors, get_cmap

viz.rmse_boxplot(rmse_draws, title="Non-linear DGP (p = 10)")
viz.inclusion_plot(res.importance, res.feature_names, top=12,
                  title="BART inclusion proportions for GDP")
print(viz.journal_table(rank, fmt="latex", caption="Table 4")) # console/html/latex

parula_colors(8)          # ['#352a87', '#0268e1', ...]  MATLAB Parula stops
get_cmap("Parula")        # a Matplotlib colormap
```

parula_colors(64)



Part VII

Guidance & take-aways

Practical guidance

Do

- Use enough trees ($m \approx 200$) and draws; too few $\Rightarrow$ noisy inclusion proportions.
- Stationarise first (Table 3 codes are built in).
- Read *inclusion proportions* for importance, not p -values.

Watch out

- BART *extrapolates* poorly beyond the training range.
- Additive-tree structure can miss very strong interactions if under-sampled.
- “Causal” here = forecasting + importance, *not* treatment effects.

Uncertainty for free

`BART.yhat_train_draws_` gives the full posterior of the fit $\Rightarrow$ 95% bands;
`sigma_draws_` gives the noise posterior.

Take-aways

-  **ARDL-BART** = single-equation distributed-lag regression with a *non-parametric* tree-ensemble mean.
-  **Simulations:** BART wins under non-linearity, is competitive under linearity — and recovers the relevant variables.
-  **Macro data:** BART forecasts best for 6 of 8 series, suggesting genuine non-linear dynamics.
-  **Reproducible:** the entire pipeline is `pip install bartardl` — theory, simulation, application, figures and tables.

“Let the data choose the shape of f .”

`pip install bartardl`

References & resources

- Mahdavi, Ehsani, Ahelegbey & Mohammadpour (2024). *Measuring Causal Effect with ARDL-BART*. Applied Mathematics **15**, 292–312. doi:10.4236/am.2024.154018.
- Chipman, George & McCulloch (2010). *BART: Bayesian Additive Regression Trees*. Ann. Appl. Stat. **4**, 266–298.
- Tibshirani (1996), LASSO; Zou & Hastie (2005), Elastic Net; Ahelegbey, Billio & Casarin (2016), Bayesian graphical VAR; Friedman (1991), MARS.



Package



github.com/merwanroudane/bartardl



pypi.org/project/bartardl

- docs: README.md + docs/guide.md

Thank you ❤️